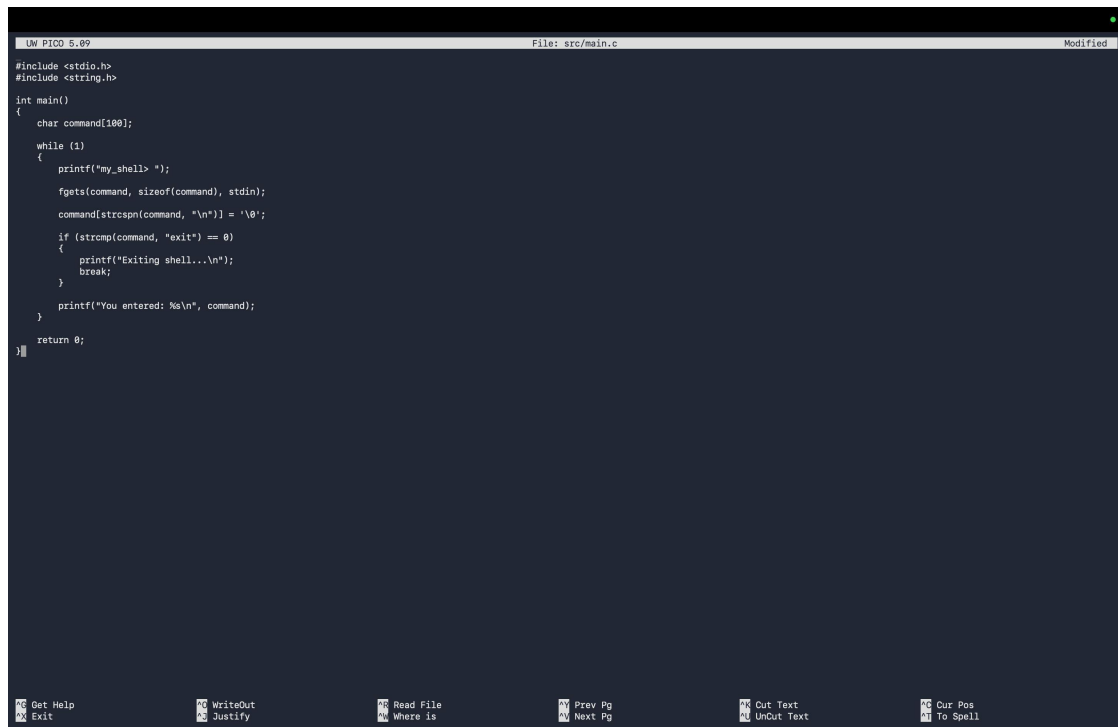


SKILL_WEEK-02 (SESSION NUMBER - 3)

Task 1: To create the main loop, display the prompt, read user input, handle exit conditions, design a control flow diagram, and test the interactive loop.

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory



The screenshot shows a code editor window titled "UW PICO 5.09" with the file "src/main.c" open. The code is a C program for a simple shell. It includes `<stdio.h>` and `<string.h>`. The `main` function starts with a `while (1)` loop. Inside the loop, it prints "my_shell> ", reads a line of input using `fgets`, and removes the trailing newline character using `command[strcspn(command, "\n")] = '\0';`. It then checks if the input is "exit" using `strcmp`. If it is, it prints "Exiting shell...\n" and breaks the loop. Otherwise, it prints "You entered: %s\n", command); and continues the loop. The program ends with `return 0;` and `}`. The editor has a dark theme and a status bar at the bottom with various icons and text.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];
    while (1)
    {
        printf("my_shell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", command);
    }
    return 0;
}
```

Figure 2: C program implementing the interactive shell main loop

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/main.c -o bin/my_shell
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of the shell program

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/my_shell
my_shell> hello
You entered: hello
my_shell> test
You entered: test
my_shell> █

```

Figure 4: Interactive shell prompt accepting user input

```

my_shell> exit
Exiting shell...
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 5: Shell exit condition

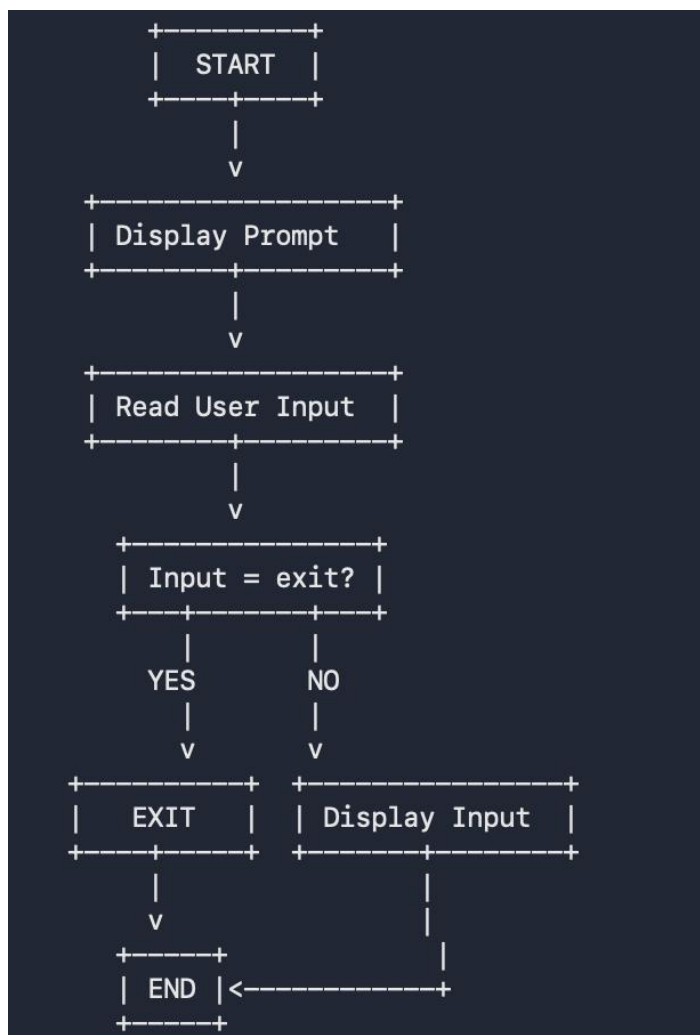


Figure 6: Control flow diagram of the interactive shell

```

[amrutha@Amruthas-MacBook-Air OSSP % git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   bin/my_shell
        deleted:    hello
        deleted:    hello.c
        modified:   src/main.c
        deleted:    src/main.c.save

```

Figure 7: Git status showing changes made to the shell main loop

```

delete mode 100644 src/main.c.save
On branch main
nothing to commit, working tree clean
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 8: Final Git status showing a clean working tree.

Result:

Escape sequences were applied successfully, command history was stored, previous and next commands were navigated correctly, and the input buffer was updated properly. Command recall functionality was tested and verified successfully.

Task 2: Using the system calls `open()`, `read()`, `write()`, and `close()`, copy the content from `File1.txt` to `File2.txt`.

```

[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP % █

```

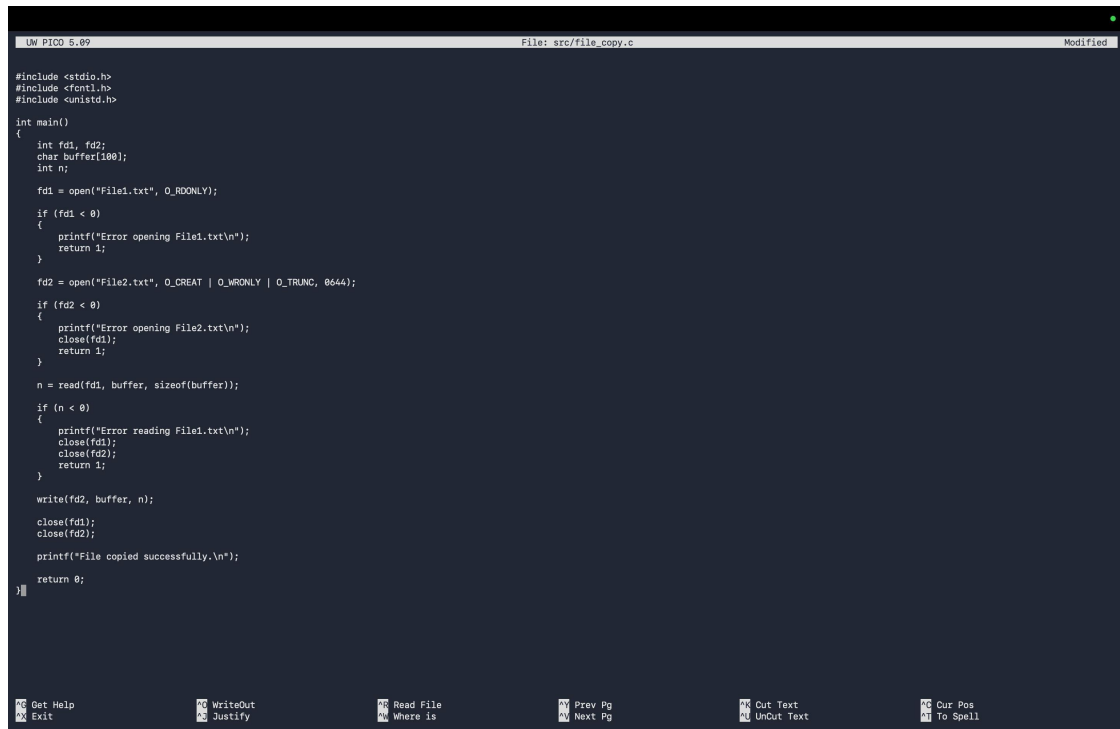
Figure 1: OSSP project directory

```

[amrutha@Amruthas-MacBook-Air OSSP % touch File1.txt
[amrutha@Amruthas-MacBook-Air OSSP % echo "This is the content of File1." > File1.txt
[amrutha@Amruthas-MacBook-Air OSSP % cat File1.txt
This is the content of File1.
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 2: Content of `File1.txt` before copying

A screenshot of a nano text editor window. The title bar shows 'Un PICO 5.09' on the left, 'File: src/file_copy.c' in the center, and 'Modified' on the right. The editor contains C code for a file copy program. The code includes headers for stdio, fcntl, andunistd, and defines a main function that opens 'File1.txt' for reading and 'File2.txt' for writing, reads data from the first file into a buffer, and writes it to the second file. The bottom status bar shows various nano editor shortcuts like 'Get Help', 'Exit', 'WriteOut', 'Justify', 'Read File', 'Where is', 'Prev Pg', 'Next Pg', 'Cut Text', 'UnOut Text', 'Cur Pos', and 'To Spell'.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0)
    {
        printf("Error opening File1.txt\n");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0)
    {
        printf("Error opening File2.txt\n");
        close(fd1);
        return 1;
    }

    n = read(fd1, buffer, sizeof(buffer));

    if (n < 0)
    {
        printf("Error reading File1.txt\n");
        close(fd1);
        close(fd2);
        return 1;
    }

    write(fd2, buffer, n);

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

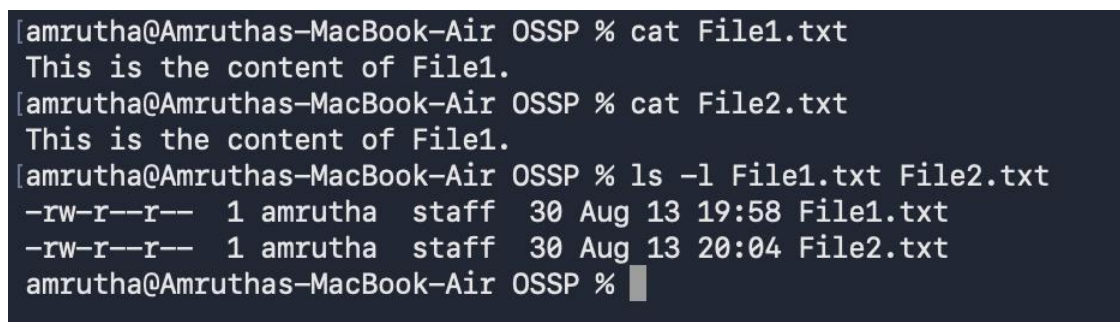
    return 0;
}
```

Figure 3: C program using open(), read(), write(), and close() system calls

A terminal window screenshot showing the steps to compile and run the file-copy program. The user is at the prompt 'amrutha@Amruthas-MacBook-Air OSSP %'. They use 'nano src/file_copy.c' to edit the file, 'gcc src/file_copy.c -o bin/file_copy' to compile it, and './bin/file_copy' to execute it. The output of the program is 'File copied successfully.'.

```
amrutha@Amruthas-MacBook-Air OSSP % nano src/file_copy.c
amrutha@Amruthas-MacBook-Air OSSP % gcc src/file_copy.c -o bin/file_copy
amrutha@Amruthas-MacBook-Air OSSP % ./bin/file_copy
File copied successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Successful compilation and execution of the file-copy program

A terminal window screenshot showing the verification of the copied content. The user runs 'cat File1.txt' and 'cat File2.txt', both showing 'This is the content of File1.'. They also run 'ls -l File1.txt File2.txt' to show file details. The output shows both files are 30 bytes and owned by amrutha staff.

```
[amrutha@Amruthas-MacBook-Air OSSP % cat File1.txt
This is the content of File1.
[amrutha@Amruthas-MacBook-Air OSSP % cat File2.txt
This is the content of File1.
[amrutha@Amruthas-MacBook-Air OSSP % ls -l File1.txt File2.txt
-rw-r--r--  1 amrutha  staff   30 Aug 13 19:58 File1.txt
-rw-r--r--  1 amrutha  staff   30 Aug 13 20:04 File2.txt
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Verification of copied content in File2.txt and File1.txt and File2.txt after the copy operation

```

amrutha@Amruthas-MacBook-Air OSSP % git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        File1.txt
        File2.txt
        bin/file_copy
        src/file_copy.c

nothing added to commit but untracked files present (use "git add" to track)
amrutha@Amruthas-MacBook-Air OSSP % git add File1.txt File2.txt src/file_copy.c bin/file_copy
amrutha@Amruthas-MacBook-Air OSSP % git commit -m "Implement file copy using system calls"
[main a319146] Implement file copy using system calls
  Committer: Amrutha <amrutha@Amruthas-MacBook-Air.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

4 files changed, 48 insertions(+)
create mode 100644 File1.txt
create mode 100644 File2.txt
create mode 100755 bin/file_copy
create mode 100644 src/file_copy.c
amrutha@Amruthas-MacBook-Air OSSP % git status
On branch main
nothing to commit, working tree clean
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 6: Git status showing files created for Task2 and clean Git status after completing Task 2

Result:

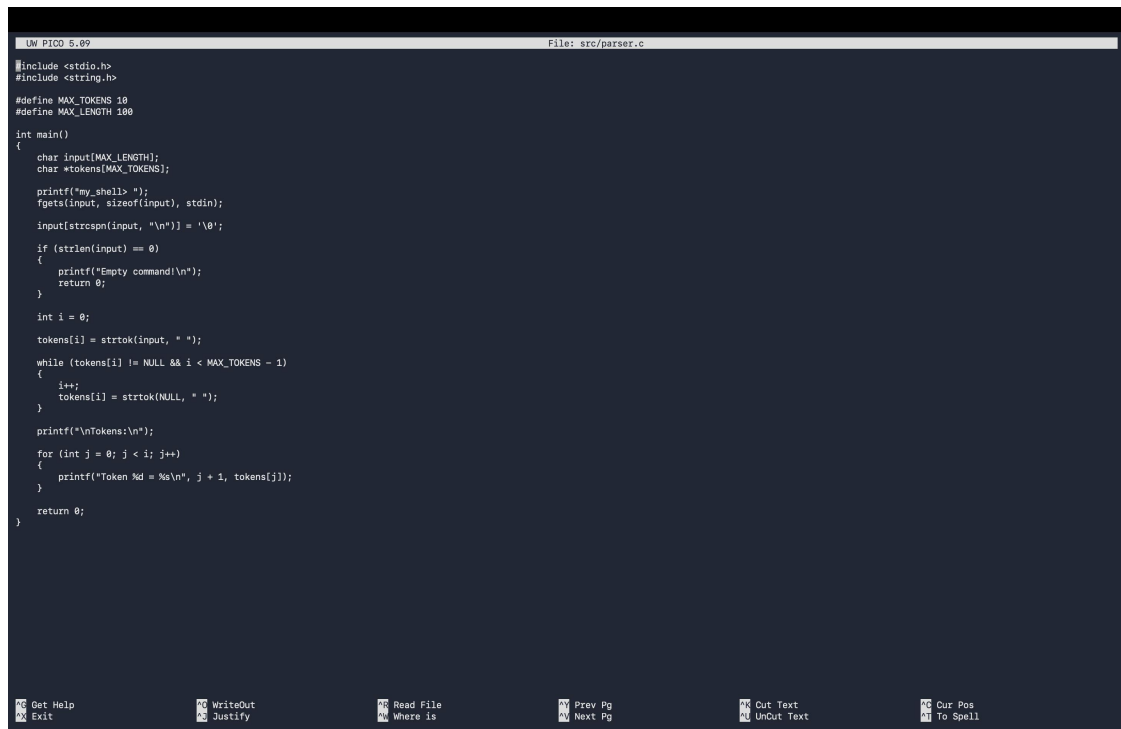
Buffers were allocated dynamically and resized successfully. Buffer overflow was prevented, linked-list memory was managed correctly, allocated memory was released properly, and memory handling was verified successfully.

(SESSION NUMBER - 4)

Task 1: To split input into tokens, identify delimiters, handle whitespace, create token structures, validate token streams, and debug parsing output.

```
Last login: Thu Aug 13 19:56:29 on ttys007
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[ pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory



```
File: src/parser.c
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 10
#define MAX_LENGTH 100

int main()
{
    char input[MAX_LENGTH];
    char *tokens[MAX_TOKENS];

    printf("my_shell> ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command\n");
        return 0;
    }

    int i = 0;
    tokens[i] = strtok(input, " ");
    while (tokens[i] != NULL && i < MAX_TOKENS - 1)
    {
        i++;
        tokens[i] = strtok(NULL, " ");
    }

    printf("\nTokens:\n");
    for (int j = 0; j < i; j++)
    {
        printf("Token %d = %s\n", j + 1, tokens[j]);
    }

    return 0;
}
```

Figure 2: C program for parsing and lexical tokenization

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/parser.c -o bin/parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of parser.c

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/parser.c -o bin/parser
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell> ls -l docs

Tokens:
Token 1 = ls
Token 2 = -l
Token 3 = docs
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Tokenization of the command `ls -l docs`

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell> mkdir project

Tokens:
Token 1 = mkdir
Token 2 = project
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Tokenization of the command `mkdir project`

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell>
Empty command!
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 6: Detection and handling of an empty command

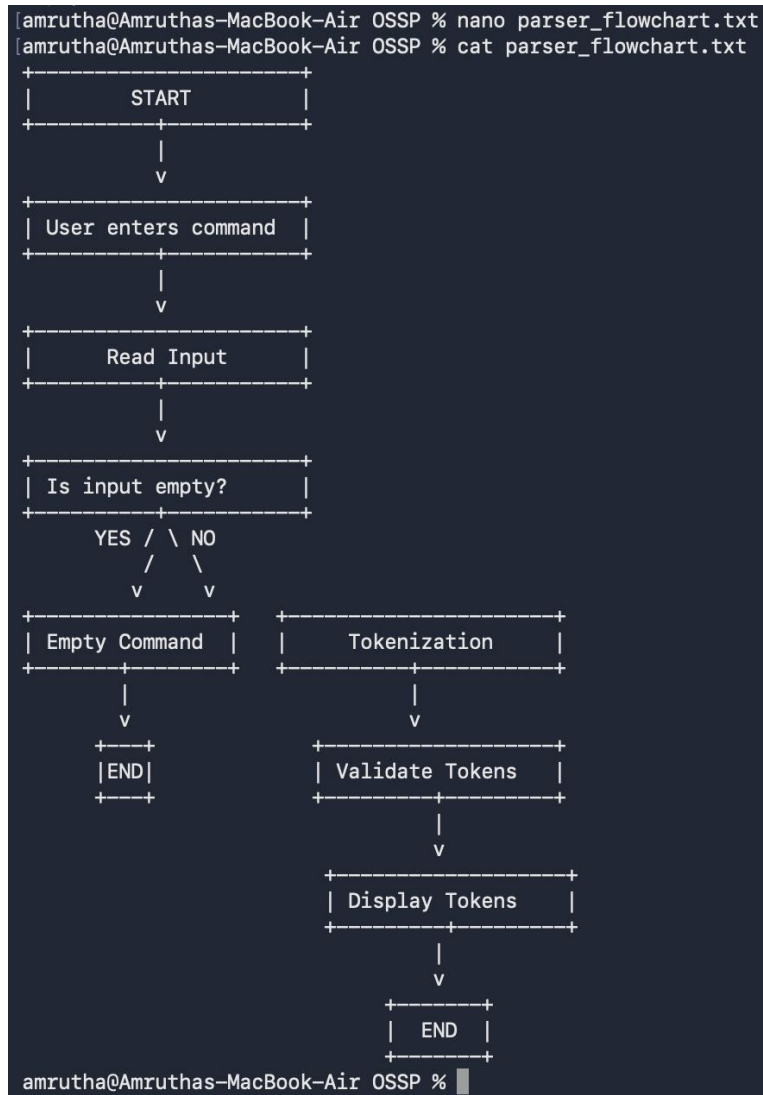


Figure 7: Flow chart for Parsing and Lexical Analysis

Task 2: To design parser logic, generate parse trees, validate syntax, detect errors, handle empty commands, and produce execution structures.

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory for Task 2

A screenshot of a code editor showing a C program named 'quote_parser.c'. The program includes standard headers like <stdio.h>, <stdlib.h>, <string.h>, <unistd.h>, <sys/types.h>, and <sys/wait.h>. It defines constants for MAX_ARGS (28), MAX_INPUT (256), and MAX_TOKEN (256). The main function 'parse_command' takes an input string and an array of arguments. It uses a loop to process the input character by character, handling single quotes and spaces to tokenize the command. The tokens are stored in an array, and the program checks for buffer overflow. The editor interface shows a menu bar with options like 'Get Help', 'Exit', 'WriteOut', 'Justify', 'Read File', 'Where is', 'Prev Pg', 'Next Pg', 'Cut Text', 'UnCut Text', 'Cur Pos', and 'To Spell'.

Figure 2: C program for single-quote parsing and command execution

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/quote_parser.c -o bin/quote_parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of quote parser

Case 1:(a)

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Child PID: 4395
Hello

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Parsing and execution of a normal command

Case 1:(b)

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Child PID: 4752
Hello World

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Single quotes preserving literal content as one argument

Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 6: Detection of unmatched single quote and syntax error

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo '$HOME'

Parsing Result:
Argument 1: echo
Argument 2: $HOME

Child PID: 5178
$HOME

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 7: Single quotes preserving literal \$HOME

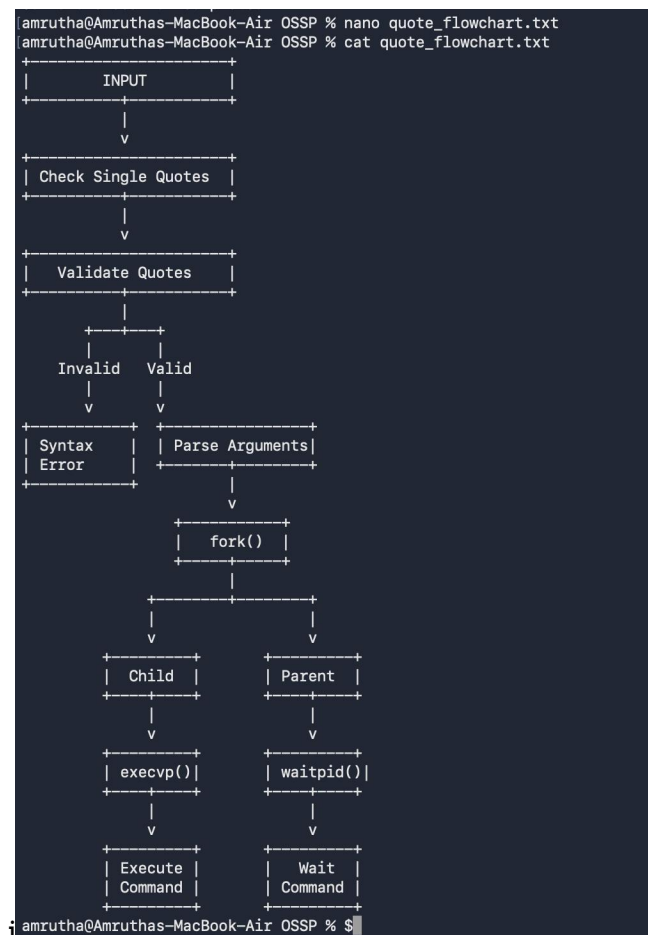


Figure 8: Terminal representation of the flowchart for single-quote parsing and command execution

Task 3: Implement double-quote handling in the shell parser to preserve spaces, allow variable expansion, parse nested tokens, validate the output, and test quoted commands.

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory for Task 4

```
int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_TOKEN];

    int argc = 0;
    int pos = 0;
    int in_double_quote = 0;
    int token_started = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        char ch = input[i];

        if (ch == '"')
        {
            in_double_quote = !in_double_quote;
            token_started = 1;
        }
        else if ((ch == ' ' || ch == '\t') && !in_double_quote)
        {
            if (token_started)
            {
                tokens[argc][pos] = '\0';
                args[argc] = tokens[argc];
                argc++;

                pos = 0;
                token_started = 0;

                if (argc >= MAX_ARGS - 1)
                    break;
            }
        }
        else if (ch == '$')
        {
            char variable[100];
            int v = 0;
            i++;

            while (input[i] != '\0' &&
                ((input[i] >= 'A' && input[i] <= 'Z') ||
                 (input[i] >= 'a' && input[i] <= 'z') ||
                 (input[i] >= '0' && input[i] <= '9') ||
                 input[i] == '.'))
            {
                variable[v++] = input[i];
                i++;
            }

            variable[v] = '\0';
            i--;

            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int j = 0; value[j] != '\0'; j++)
                {
                    if (pos < MAX_TOKEN - 1)
                        tokens[argc][pos++] = value[j];
                }
            }
        }
    }

    tokens[argc][pos] = '\0';
    args[argc] = tokens[argc];
    argc++;

    return argc;
}
```

Figure 2(a): Double-quote parsing program – input and quote validation

```
    }
    token_started = 1;
}
else
{
    if (pos < MAX_TOKEN - 1)
        tokens[argc][pos++] = ch;

    token_started = 1;
}
}

if (in_double_quote)
{
    printf("Syntax Error: Unmatched double quote.\n");
    return -1;
}

if (token_started)
{
    tokens[argc][pos] = '\0';
    args[argc] = tokens[argc];
    argc++;
}

args[argc] = NULL;
return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;
}
```

Figure 2(b): Double-quote parsing program – variable expansion and tokenization

```

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("Empty command.\n");
    return 0;
}

int argc = parse_command(input, args);

if (argc < 0)
    return 1;

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    printf("\nChild Process\n");
    printf("PID = %d\n\n", getpid());

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}
else
{
    waitpid(pid, NULL, 0);
    printf("\nChild process completed.\n");
}

return 0;
}
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 2(c): Double-quote parsing program – process creation and command execution

Case 1:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/double_quote_parser.c -o bin/double_quote_parser
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello amrutha

Child Process
PID = 18958

Hello amrutha

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of double-quote parser and Double quotes preserving spaces and expanding the \$USER variable

Case 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Detection of unmatched double quote

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 19967

Hello World

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Double quotes preserving spaces as a single argument

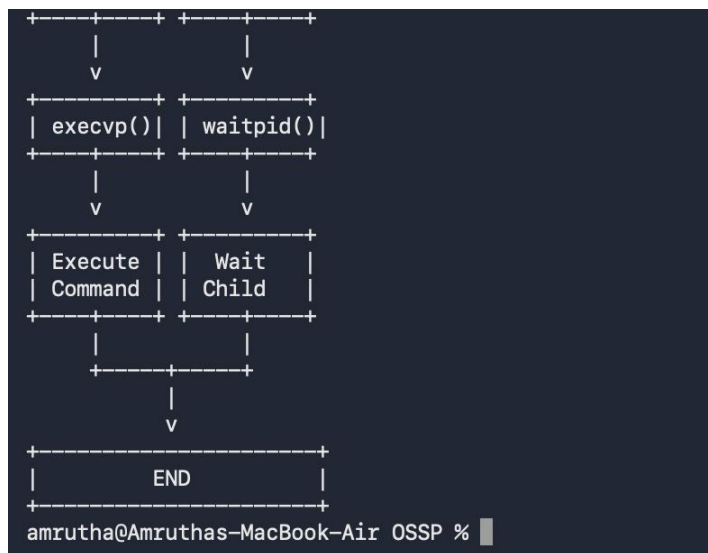
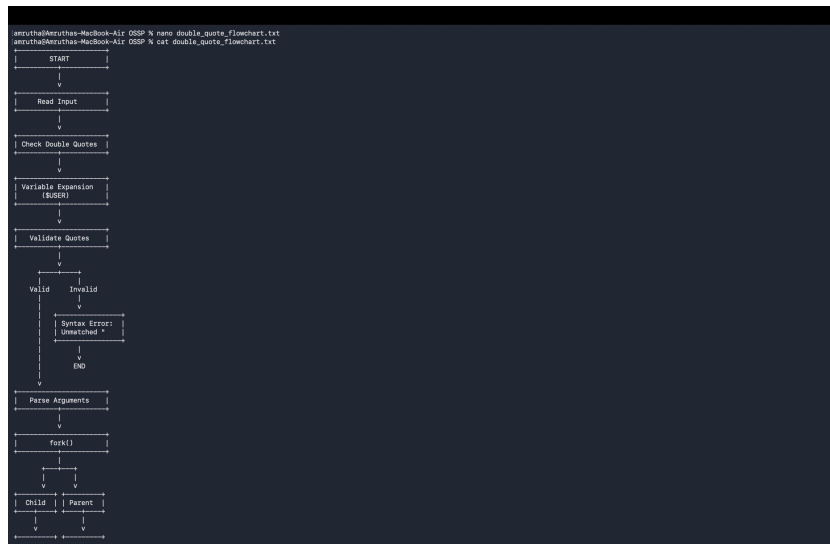


Figure 6: Terminal flowchart for double-quote validation, variable expansion and command execution

```

amrutha@Amruthas-MacBook-Air OSSP % git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/parser.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bin/double_quote_parser
        bin/parser
        bin/quote_parser
        double_quote_flowchart.txt
        parser_flowchart.txt
        quote_flowchart.txt
        src/double_quote_parser.c
        src/parser.c.save
        src/quote_parser.c

no changes added to commit (use "git add" and/or "git commit -a")
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 7: Git status after implementing double-quote parsing

Task 4: Implement escape-sequence handling in the shell parser to handle escaped spaces and special symbols, preserve characters correctly, validate the parser output, and test complex input cases.

```
[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
```

Figure 1: OSSP project directory for Task 4

```
int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_TOKEN];

    int argp = 0;
    int pos = 0;
    int escaped = 0;
    int token_started = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        char ch = input[i];

        if (escaped)
        {
            if (pos < MAX_TOKEN - 1)
                tokens[argp][pos++] = ch;

            escaped = 0;
            token_started = 1;
        }
        else if (ch == '\\')
        {
            escaped = 1;
            token_started = 1;
        }
        else if (ch == ' ' || ch == '\t')
        {
            if (token_started)
            {
                tokens[argp][pos] = '\0';
                argp[argp++] = tokens[argp];
                pos = 0;
                token_started = 0;

                if (argp >= MAX_ARGS - 1)
                    break;
            }
        }
        else
        {
            if (pos < MAX_TOKEN - 1)
                tokens[argp][pos++] = ch;

            token_started = 1;
        }
    }

    if (escaped)
    {
        printf("Syntax Error: Trailing escape character.\n");
        return -1;
    }

    if (token_started)
    {
        tokens[argp][pos] = '\0';
        argp[argp++] = tokens[argp];
    }
}
```

Figure 2(a): Escape sequence parser – input processing and escape handling

```
    return argp;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    int argp = parse_command(input, args);

    if (argp < 0)
        return 1;

    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < argp; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID = %d\n", getpid());

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }
    else
    {
        waitpid(pid, NULL, 0);
        printf("\nChild process completed.\n");
    }

    return 0;
}

[amrutha@Amruthas-MacBook-Air OSSP %]
```

Figure 2(b): Escape sequence parser – tokenization and validation and process creation and command execution

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/escape_parser.c -o bin/escape_parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of escape sequence parser

Case 1:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 46872

Hello World

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Case 1 – Handling escaped spaces as a single argument

Case 2:

```
Child process completed.
amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\!

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!

Child Process
PID = 47192

Hello!

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Case 2 – Preserving an escaped special character

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo My\ Project\ Folder

--- Parsed Arguments ---
Argument 1: echo
Argument 2: My Project Folder

Child Process
PID = 47541

My Project Folder

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 6: Parsing multiple escaped spaces in a complex command.

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\
Syntax Error: Trailing escape character.
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 7: Detection of an invalid trailing escape character

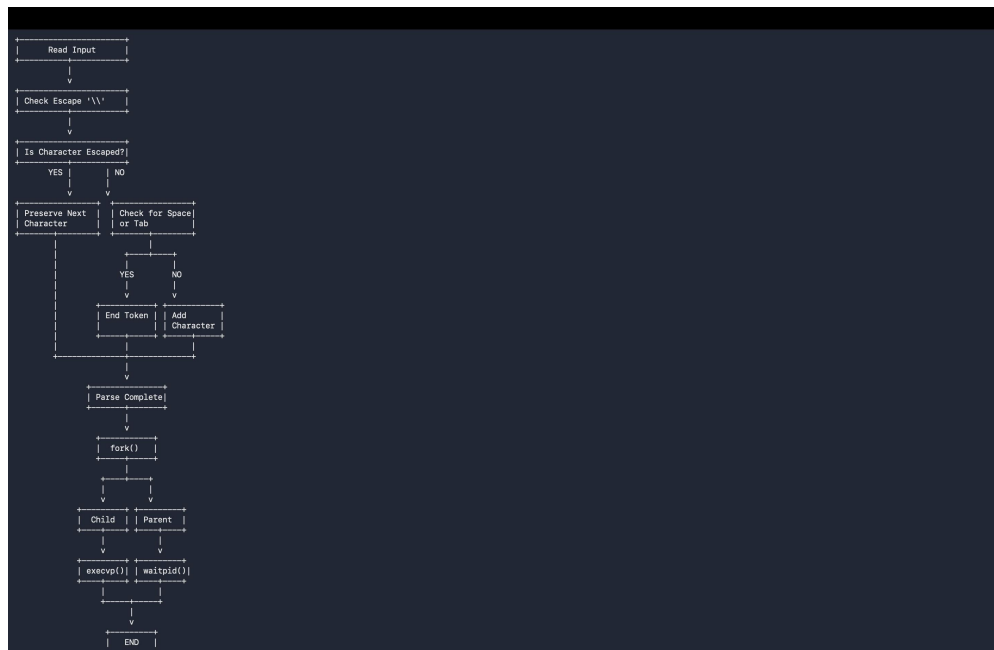


Figure 8: Terminal flowchart for escape sequence parsing and command execution

Task 5: Implement child-process creation and command execution using `fork()` and `execvp()`, pass command-line arguments correctly, handle execution errors, manage the parent process using `waitpid()`, and test command launching.

```
[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory.

```
[amrutha@Amruthas-MacBook-Air OSSP % cat src/process_launcher.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    char argument[100];

    printf("myshell$ ");

    scanf("%99s", command);

    if (scanf("%99s", argument) == 1)
    {
        /* Argument is provided */
    }
    else
    {
        argument[0] = '\0';
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
}
```

Figure 2(a): Process launcher program using `fork()` and `execvp()`

```

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        if (argument[0] != '\0')
        {
            char *args[] = {command, argument, NULL};
            execvp(command, args);
        }
        else
        {
            char *args[] = {command, NULL};
            execvp(command, args);
        }

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }

    return 0;
}
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 2(b): Parent process management using waitpid() and exit-status handling

Case 1:

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshe11$ ls
Parent Process PID: 91668
Created Child PID: 91687
Child Process PID: 91687
assets          double_quote_flowchart.txt  File2.txt        Makefile        quote_flowchart.txt  src
bin             escape_flowchart.txt       include          obj             README.md           tests
docs            File1.txt                  LICENSE          parser_flowchart.txt  scripts
Child exited with status: 0
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 3: Case 1 – Successful execution of the ls command using a child process

Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshell$ ls @l
Parent Process PID: 92124
Created Child PID: 92140
Child Process PID: 92140
ls: @l: No such file or directory
Child exited with status: 1
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Case 2 – Handling an invalid argument and execution error

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshell$ ls src
Parent Process PID: 92461
Created Child PID: 92477
Child Process PID: 92477
builtin.c      escape_parser.c      file_copy.c      parser.c      process_launcher.c  shell.c
double_quote_parser.c  executor.c      main.c      parser.c.save  quote_parser.c
Child exited with status: 0
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Passing an argument to the child process and executing the ls command

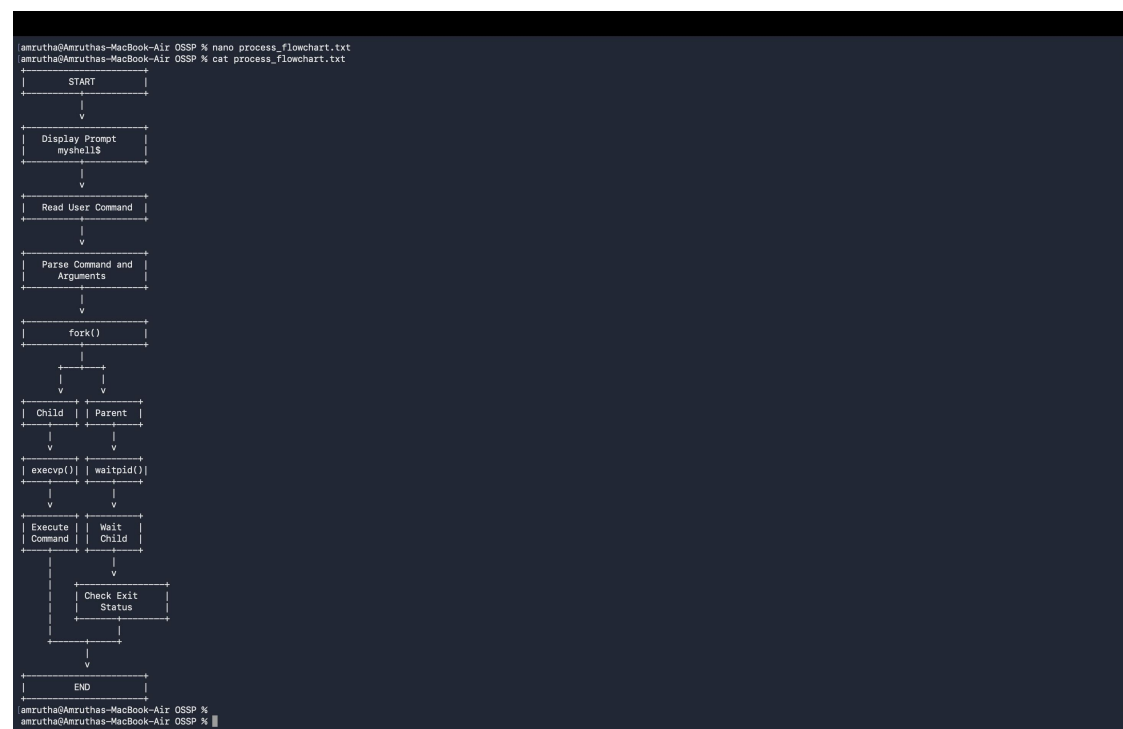


Figure 6: Terminal flowchart for process creation, command execution and parent process m

